Fellowes

Writing Object Oriented Elixir Architecture

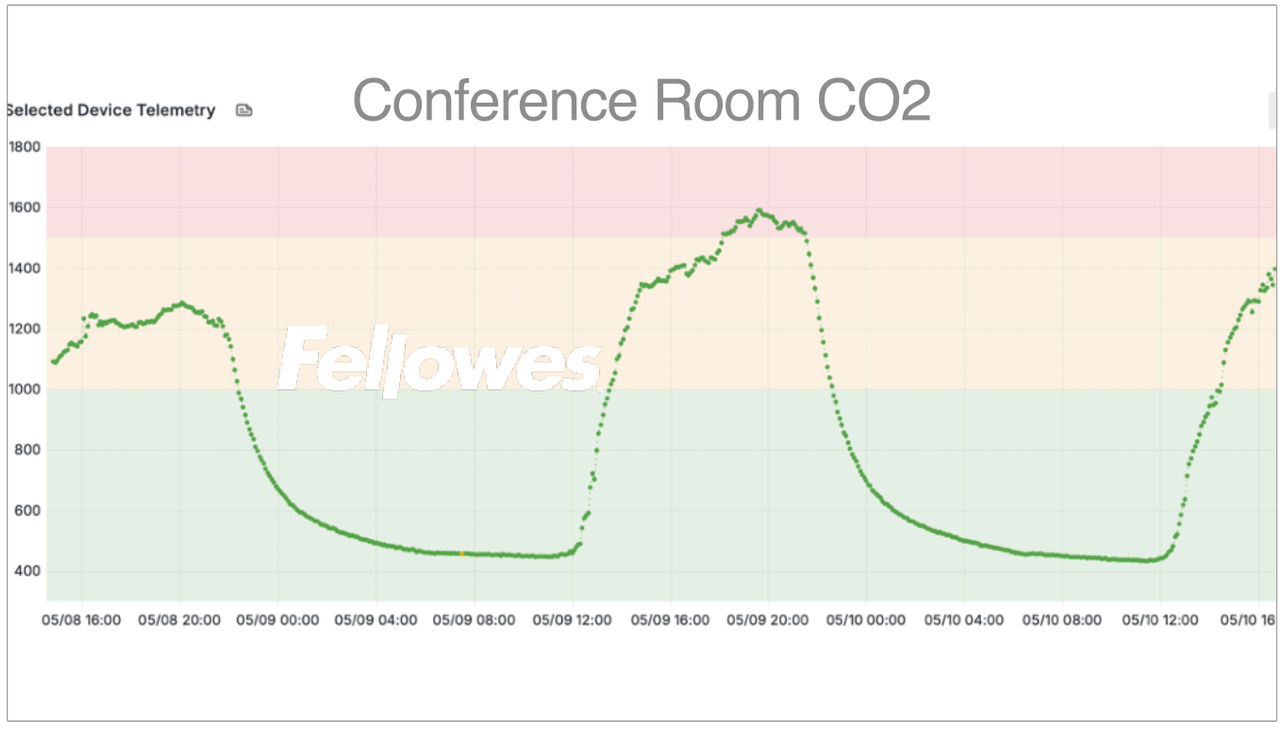


Taun Chapman

Fellowes Staff Engineer

Goal:

Motivate you to continue to create improved tools for software development in the Elixir ecosystem by giving you as yet unrealized concrete examples from the past.


Selected Device Telemetry
Conference Room CO2
1800
1600
1400
1200
1000
800
600
400
Fellowes
05/08 16:00
05/08 20:00
05/09 00:00
05/09 04:00
05/09 08:00
05/09 12:00
05/09 16:00
05/09 20:00
05/10 00:00
05/10 04:00
05/10 08:00
05/10 12:00
05/10 16

We are already OO Programming in Elixir
when we use best practices.

Anyone can write bad code.
There is no need for me to give that talk.

In my opinion Erlang/Elixir/Pony etc are the only languages which can be called OO.

Joe Armstrong

Yes, Erlang is in many ways much more like the original Smalltalk, and Java (and C++) are much further away from the idea.

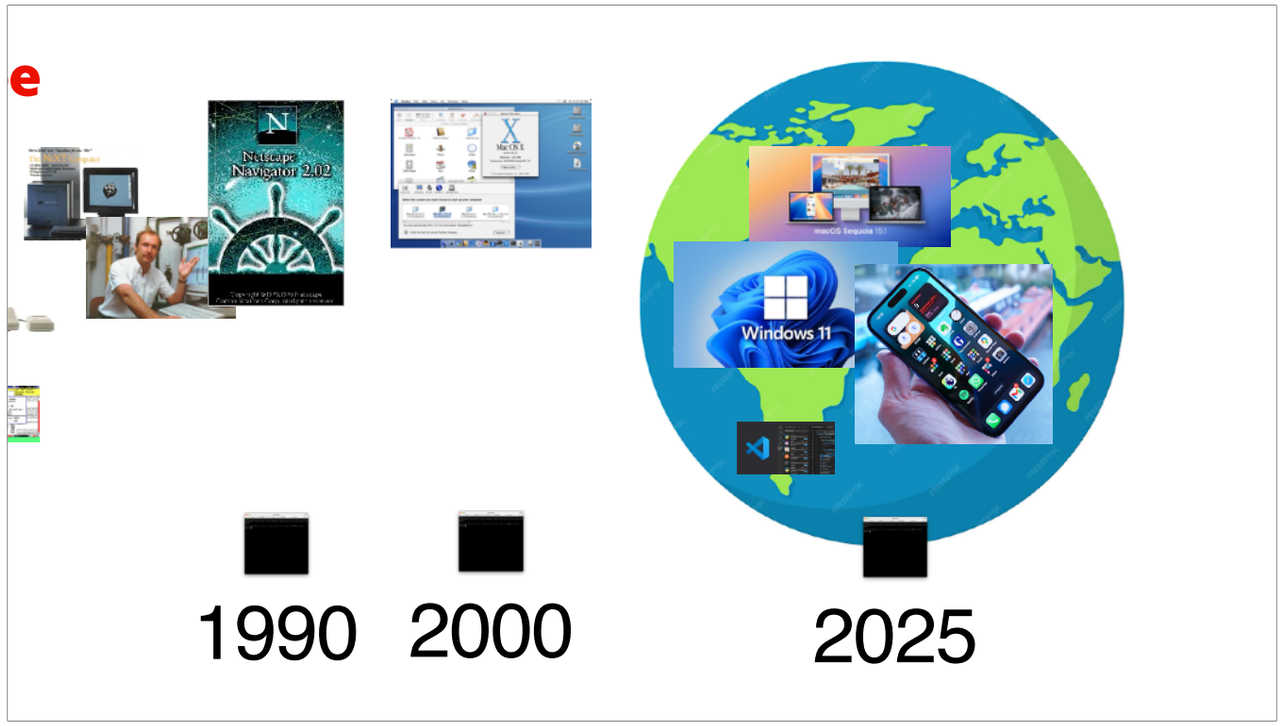
I thought of objects being like biological cells and/or individual computers on a network, only able to communicate with messages.

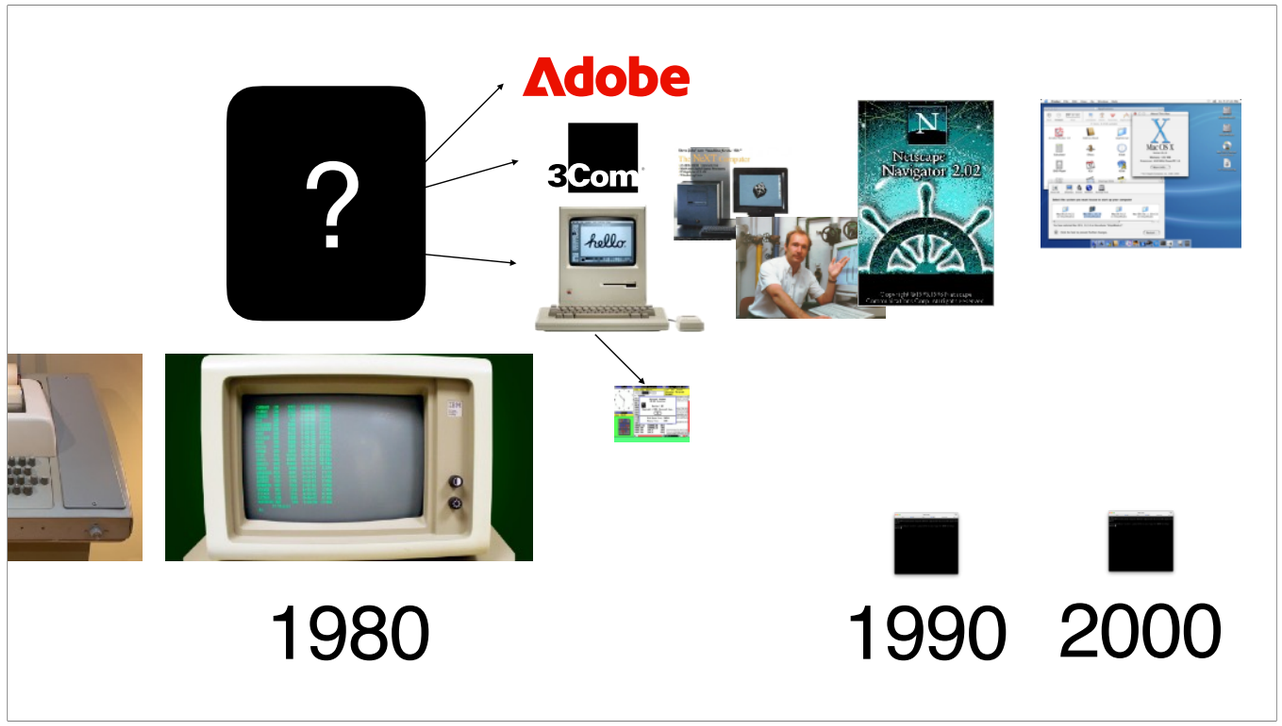
Alan Kay

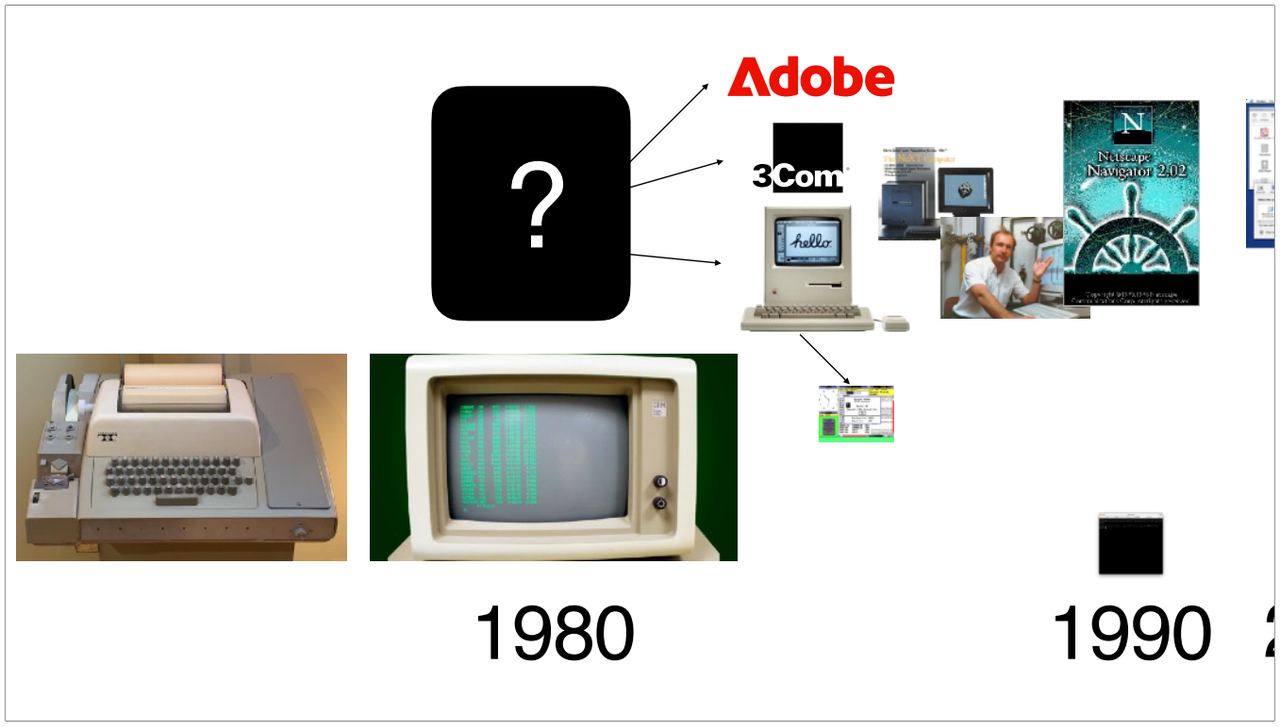


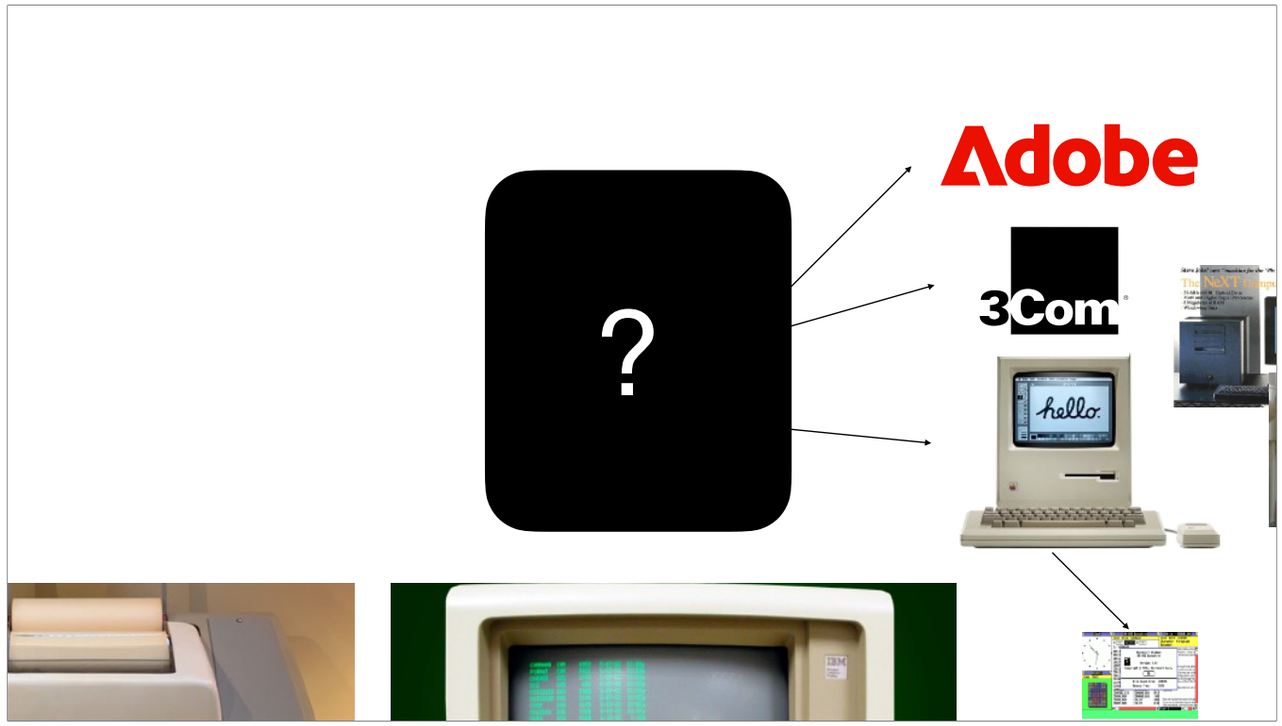
2025

Who runs the world?

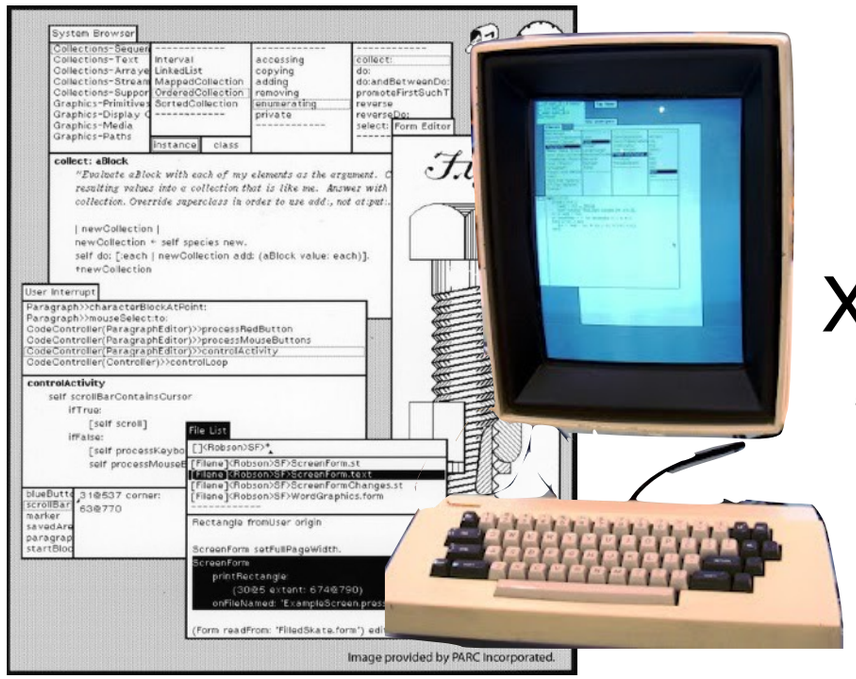

Netscape Navigator 2.02
Windows 11
1990
2000
2025


Adobe
?
3Com
hello
Netscape Navigator 2.02
1980
1990
2000

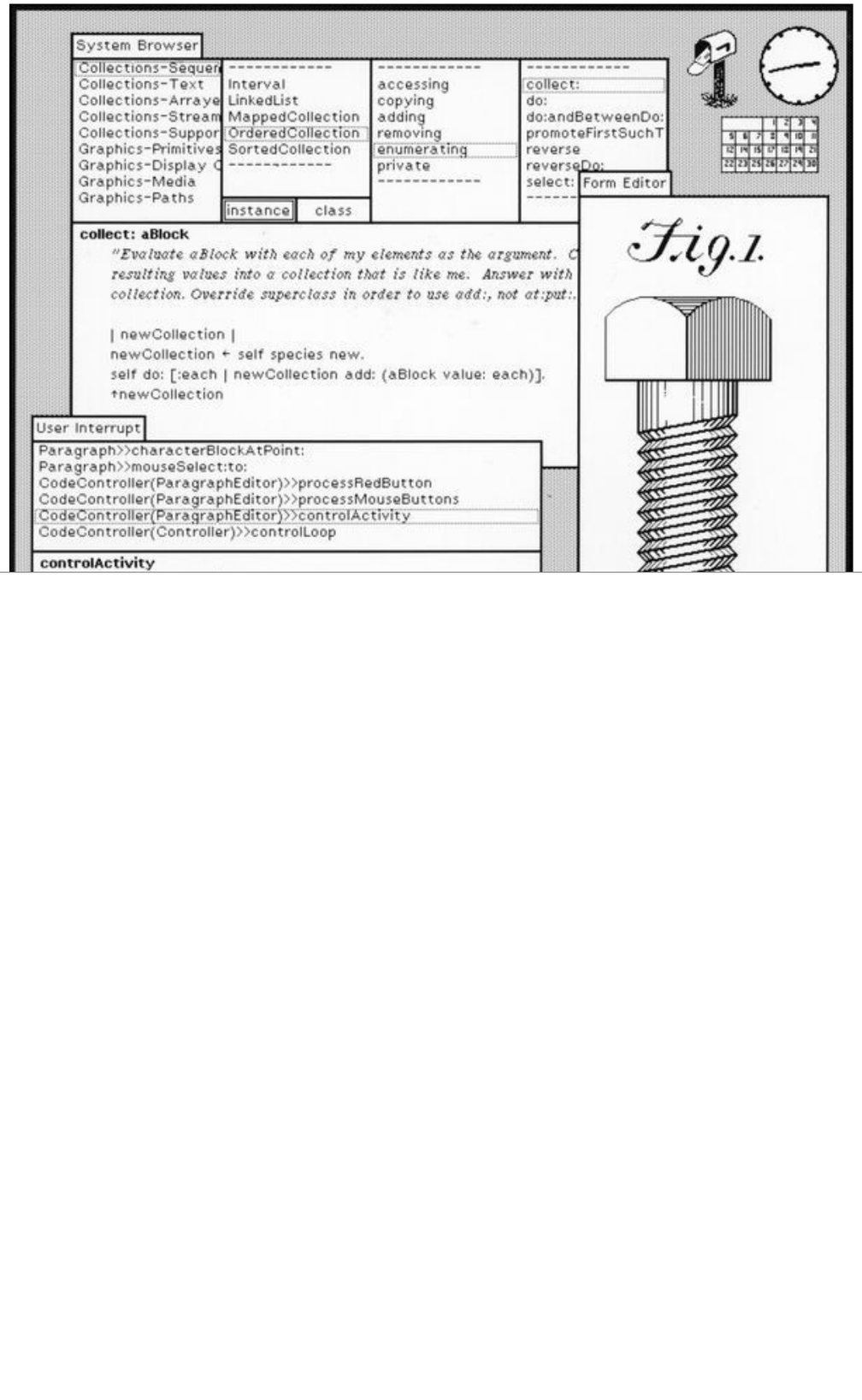

Adobe
3Com
hello
Netscape
Navigator 2.02
1980
1990


?
Adobe
3Com
hello.

1980



Xerox Alto


System Browser
Collections-Sequer
Collections-Text
Collections-Arraye
Collections-Stream
Collections-Suppor
Graphics-Primitives
Graphics-Display C
Graphics-Media
Graphics-Paths
Interval
LinkedList
MappedCollection
OrderedCollection
SortedCollection
instance
class
accessing
copying
adding
removing
enumerating
private
collect:
do:
do:andBetweenDo:
promoteFirstSuchT
reverse
reverseDo:
select:
collect: aBlock
"Evaluate aBlock with each of my elements as the argument. C
resulting values into a collection that is like me. Answer with
collection. Override superclass in order to use add:, not at:put:.
| newCollection |
newCollection ← self species new.
self do: [:each | newCollection add: (aBlock value: each)].
↑newCollection
User Interrupt
Paragraph>>characterBlockAtPoint:
Paragraph>>mouseSelect:to:
CodeController(ParagraphEditor)>>processRedButton
CodeController(ParagraphEditor)>>processMouseButtons
CodeController(ParagraphEditor)>>controlActivity
CodeController(Controller)>>controlLoop
controlActivity
Form Editor
Fig.1.

Demo

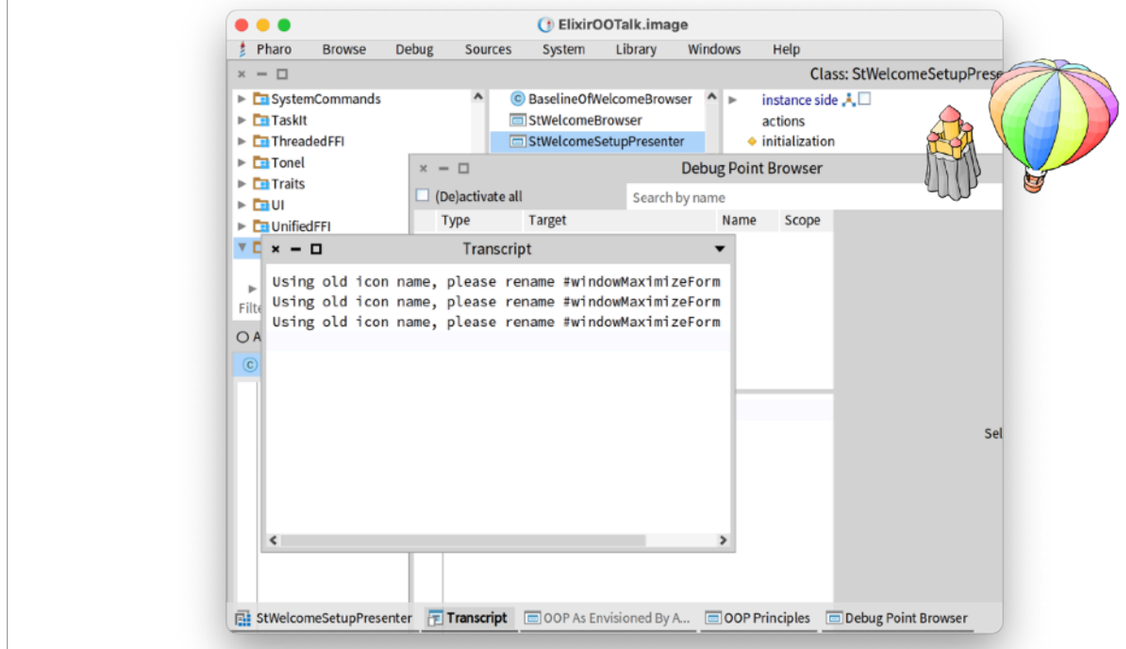


Smalltalk visibility demo

- Original IDE? Editor helps with organization and encapsulation. Can run code within the editor. **Every editor window is an iex console**. Includes documentation view.
- Inspector! Like all of our console command line introspection or Observer.
- Live debugger - Ability to call any object message while running
- Dynamic - Live changes
- No waiting for compilation
- Consistently won timed programming contests

Demo

- The environment IS Smalltalk
- Every pane is a potential code editor+compiler+linker+hotswap
- Code viewing is a virtual view - not limited by the file layout.
- Debugger - Rollback to previous stack position



There is no relation between code and it’s location in a file.
The Tool, manages the organization for you. Much better than VSCode does.
Code is compiled in the editor.

Inspirations from Smalltalk

- Code as prose - be willing to spell it out.
- Code that is just there for me - visibility functions, helper functions aka tools
- Use “setters” - easier code completion, easier debugging, more visibility
- Code as action - “increment_user_count/2” vs “cnt = cnt++1”
- Keep it small stupid - all the relevant code should always be visible.
  - No don’t just get a bigger screen.
- Decomposition & Encapsulation - Don’t be afraid to break big problems down into smaller problems and treat them as a building block.
- We could use more tools that improve our observability, and iterative speed

When I looked into the Fellowes Nerves code base, it was obvious an embedded C engineer had been writing Elixir as C. Short
When I talk about using OOP principles in Elixir, I am talking about the following.

Elixir

How OOP principles and Erlang support visibility

- Erlang is pretty unique in having a very similar feature set.
- Encapsulation is like a test point
- Decomposition - improves testability. Testing is a way of observing.
- Introspection - Observer, children, state
- Dynamic - pasting code in console, hot updates
- LiveBook - a web based “playground”

But …

- We still use dbg(), IO.inspect, etc ~ printf
- We still constantly context switch between writing, running, evaluating, writing, …
- We are still managing our text files and saving syntactically broken code.

How Can We …

Take inspiration from Smalltalk
To make our tools more user friendly

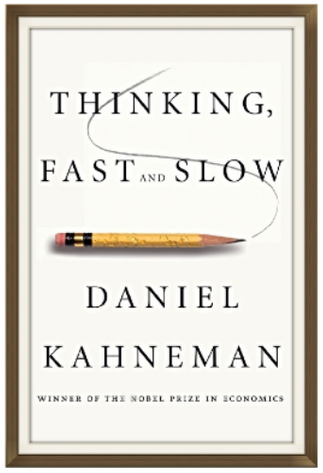
- A view which both renders the stack or a sequence of calls but **also** makes it trivial to use the view to move up and down the call chain
    - A tool to show inferred call graphs? Pick a start point, …
    - Mix deps.tree in LiveBook? Phoenix with links + expand/ collapse node?
- A truly simplified view of Action->A->B->C->D as a hyperlinked list view with a pane for the code but just the function? That is editable? Maybe even a view that shows each function in an editor all at once as if they were in the same file? (**Give an example image from Fellowes code**). Like Trace but truly for Elixir with a Phoenix UI?
- Message histories? Maybe a library which can be plugged into a GenServer message box? Like Erlang et()? Example?

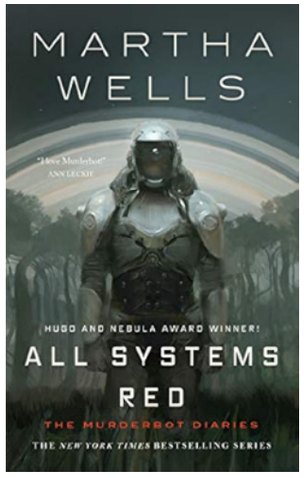
… make what we do easier?

- What if unit_test tests could be like Smalltalk debugger. Pause, edit, save, retry test. Low hanging? Smalltalk inspector is basically a GUI stack inspector with a code editor/compiler, etc. Like LiveBook but more.
- An observer that works? No Wx issues. Maybe via Phoenix like the PhoenixDashboard but more. And shows the last message rather than no messages because the queue is empty?
- LiveBook + Trace + Editor + File ingester <- too much?

- Your ideas here …

Taun's Books of 2024





Thank You

Fellowes